

BACKEND FRAMEWORKS · DJANGO REST FRAMEWORK

Autenticação e Permissões

Protegendo APIs com JWT, Users, Groups e Permissions

PROJETO: PERSONAL JOURNAL API

O QUE VAMOS CONSTRUIR

Uma API de diário pessoal onde cada usuário só pode ver, editar e excluir suas próprias anotações.

Cadastro, login com JWT, rotas protegidas, permissões por dono e grupos de usuários (autor / leitor).

Agenda — o que vamos ver hoje

01

Conceitos de Segurança

Por que proteger uma API e diferença entre autenticar e autorizar.

02

JWT na prática

O que é um token, como funciona, anatomia (header.payload.signature).

03

Setup do projeto

Journal API — Django + DRF + SimpleJWT + apps accounts e journal.

04

Registro e Login

Criar usuário, gerar token de acesso e refresh, testar no Postman.

05

Permissões DRF

IsAuthenticated, IsOwnerOrReadOnly e regras de acesso por usuário.

06

Groups e segurança

Grupos de permissão, boas práticas e desafios para casa.

DE ONDE PARAMOS

Relembrando: API REST com DRF

Na aula passada construímos a Task Manager API: um CRUD completo com Django REST Framework. Hoje vamos a um próximo passo natural — proteger essa API para que cada usuário só veja seus próprios dados.

Model

Representa a tabela no banco. Cada classe vira uma tabela via ORM.

Serializer

Traduz objeto Python ↔ JSON e valida dados de entrada.

ViewSet

Uma classe que agrupa todas as ações CRUD do recurso.

Router

Gera as URLs automaticamente a partir do ViewSet.

PROBLEMA

A Task API que fizemos é pública. Qualquer pessoa que acessa `/api/tasks/` vê e edita as tarefas de todo mundo. Como fazer com que cada usuário tenha seu próprio espaço, e só ele consiga mexer nos próprios dados?

Por que proteger uma API?

Uma API exposta na internet é uma porta aberta. Sem proteção, qualquer pessoa pode ler, alterar ou apagar dados. Pense em 3 cenários:



Banco

Sem autenticação, qualquer cliente conseguiria ver o saldo de outro cliente — ou pior, transferir dinheiro.



Email

Imagina abrir o Gmail e ver a caixa de entrada de outra pessoa. A API precisa saber quem está perguntando.



Journal pessoal

Nosso projeto de hoje. Cada usuário tem suas anotações privadas. Ninguém mais pode ler nem editar.

EM RESUMO

Toda API que lida com dados de usuários precisa responder duas perguntas: "quem é você?" (autenticação) e "o que você pode fazer?" (autorização).

Autenticação vs Autorização

Dois conceitos distintos que sempre andam juntos. Confundir os dois é o erro mais comum de quem começa em segurança de APIs.

Authentication

Autenticação (AuthN)

"Quem é você?"

Provar a identidade do usuário. O cliente envia credenciais (email/senha, token) e o servidor confirma quem ele é.

EXEMPLO

Login com email e senha → recebe um JWT

Authorization

Autorização (AuthZ)

"O que você pode fazer?"

Depois de saber quem é o usuário, decidir o que ele tem permissão para acessar ou modificar.

EXEMPLO

Usuário comum não pode acessar /admin/

Tipos de autenticação no DRF

O DRF suporta vários esquemas. Cada um tem trade-offs entre simplicidade, segurança e escalabilidade.

TIPO	COMO FUNCIONA	QUANDO USAR
BasicAuthentication	Envia email e senha em todo request (base64)	Apenas testes locais. Inseguro em produção.
SessionAuthentication	Usa o sistema de sessão do Django (cookie)	Frontend e backend no mesmo domínio.
TokenAuthentication	Um token fixo por usuário no banco	Apps simples. Não expira sozinho.
JWT (SimpleJWT)	Token assinado, com expiração, sem estado	Apps modernas, mobile, SPAs. Padrão atual.

→ Nesta aula vamos usar **JWT (SimpleJWT)** — o padrão moderno para APIs REST.

JWT — JSON Web Token

É um token assinado digitalmente, contendo informações sobre o usuário, que o servidor entrega após o login. O cliente envia esse token em toda requisição subsequente para provar quem ele é — sem precisar enviar a senha de novo.

1

POST /login

Cliente envia email + senha

2

Token gerado

Servidor valida e devolve JWT

3

Authorization

Cliente guarda e reenvia em cada request

4

Validação

Servidor verifica assinatura sem ir ao DB

✓ Vantagens

- Sem estado — o servidor não guarda sessão
- Funciona bem em apps mobile e SPAs
- Pode carregar dados do usuário no payload
- Expira automaticamente (controle de tempo)

⚠ Cuidados

- Não dá para "invalidar" antes de expirar
- Nunca colocar dados sensíveis no payload
- Sempre usar HTTPS — token vai no header
- A SECRET_KEY do projeto precisa ser forte

Anatomia de um JWT

Um JWT é uma string com 3 partes separadas por pontos: header.payload.signature. Cada parte é um JSON codificado em Base64URL.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

• eyJ1c2VyX2lkIjoxLCJleHAiOjE3MzAwMDAwMDB9

• aBcDeF1234XyZ789-signature-hash

Header

Tipo do token e algoritmo de assinatura usado.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

As "claims" — informações do usuário e metadados.

```
{
  "user_id": 1,
  "username": "wendell",
  "exp": 1730000000
}
```

Signature

Hash do header + payload + SECRET_KEY. Garante que o token não foi alterado.

```
HMACSHA256(
  base64(header) + "." +
  base64(payload),
  SECRET_KEY
)
```


PROJETO DA AULA

Personal Journal API

Uma API de diário pessoal onde cada usuário só pode ver, editar e apagar suas próprias entradas.

O que vamos construir nesta aula:



Registro & Login

Endpoints para criar conta e receber JWT



CRUD de Entradas

Criar, listar, editar e apagar diary entries



Permissões

Cada user só acessa o próprio diário



Grupos

Editor e Leitor com permissões diferentes

Endpoints que vamos criar:

POST /api/auth/register/ • POST /api/auth/login/ • POST /api/auth/refresh/
GET POST /api/entries/ • GET PUT PATCH DELETE /api/entries/{id}/

Preparando o ambiente virtual

Sempre comece um projeto Django criando uma virtualenv isolada. Isso evita conflito de versões entre projetos e mantém suas dependências organizadas.

TERMINAL • bash / powershell

```
# 1. Crie a pasta do projeto e entre nela
mkdir personal-journal-api
cd personal-journal-api

# 2. Crie a virtualenv
python -m venv venv

# 3. Ative a virtualenv
# Windows (PowerShell):
venv\Scripts\Activate.ps1
# Linux / macOS:
source venv/bin/activate
```

Dica do professor

Quando a venv está ativada, você verá (venv) no início da linha do terminal. Se não aparecer, o comando `python pip` vai instalar no Python global — repita o passo 3 antes de continuar.

Instalando dependências e criando os apps

Vamos instalar 3 pacotes: o Django, o DRF e o djangorestframework-simplejwt (que vai gerar e validar nossos tokens JWT). Em seguida criamos o projeto e dois apps: accounts (registro/login) e journal (entradas do diário).

TERMINAL · (venv) ativada

```
# 1. Instalar dependências
pip install django
pip install djangorestframework
pip install djangorestframework-simplejwt

# 2. Criar o projeto Django (atenção ao ponto no final!)
django-admin startproject core .

# 3. Criar os dois apps
python manage.py startapp accounts
python manage.py startapp journal

# 4. Conferir se está tudo certo – deve abrir no http://127.0.0.1:8000
python manage.py runserver
```

PASSO 3 · CONFIGURAÇÃO

Configurando o settings.py

Abra core/settings.py e adicione os apps + a configuração do DRF e do Simple JWT. Isso diz ao Django que toda requisição autenticada deve passar pelo validador de JWT.

 core/settings.py

```
from datetime import timedelta

INSTALLED_APPS = [
    # ...defaults do Django...
    'rest_framework',
    'rest_framework_simplejwt',
    'accounts',
    'journal',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
}
```

Modelando a entrada do diário

A chave deste projeto é o campo `author` — uma `ForeignKey` para `User`. É ele que vai amarrar cada entrada ao seu dono e permitir a regra "só vejo o que é meu".

journal/models.py

```
from django.db import models
from django.contrib.auth.models import User

class JournalEntry(models.Model):
    MOOD_CHOICES = [
        ('happy', '😄 Feliz'),
        ('neutral', '😐 Neutro'),
        ('sad', '😞 Triste'),
    ]

    author = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name='entries'
    )
    title = models.CharField(max_length=120)
    content = models.TextField()
    mood = models.CharField(
        max_length=10,
        choices=MOOD_CHOICES,
        default='neutral'
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']
```

Campos explicados

- **author**
Quem é o dono — vai puxar do JWT em toda request
- **title**
Título curto da entrada (até 120 caracteres)
- **content**
Texto longo, sem limite — o diário em si
- **mood**
Humor com choices (3 opções fixas)
- **created_at**
Data automática na criação (`auto_now_add`)

O Django já vem com tudo de usuário

Antes de criarmos rotas de cadastro e login, vale entender o que o Django já entrega de graça pelo `django.contrib.auth`.

MODEL

User

Modelo pronto com `username`, `email`, `password` (criptografada), `first_name`, `last_name`, `is_active`, `is_staff` e mais.

```
from django.contrib.auth.models import
User
```

AGRUPAMENTO

Group

Coleções nomeadas de usuários — ex.: "Leitores", "Editores", "Admins". Você atribui permissões ao grupo, não ao usuário um a um.

```
user.groups.add(grupo)
```

AUTORIZAÇÃO

Permission

Cada model gera 4 automáticas: `add`, `change`, `delete`, `view`. Você também pode criar permissões customizadas no Meta do model.

```
user.has_perm('journal.add_journalentr
y')
```

Aplicar migrations e criar o admin

1

Gerar migrations

O Django lê os models (JournalEntry + auth) e gera arquivos SQL versionados em journal/migrations/.

\$ **TERMINAL**

```
python manage.py makemigrations
```

2

Aplicar no banco

Cria as tabelas no SQLite — incluindo auth_user, auth_group, auth_permission e journal_journalentry.

\$ **TERMINAL**

```
python manage.py migrate
```

3

Criar superusuário

Cria o primeiro usuário com acesso total ao admin. Vai pedir nome, e-mail e senha — anote!

\$ **TERMINAL**

```
python manage.py createsuperuser
```



Dica: o superuser também é um User normal. Você pode fazer login com ele no JWT depois — não precisa criar outro só para testar.

RegisterSerializer — criando o usuário

Não dá para cadastrar um User direto pelo ModelSerializer — a senha precisa ser criptografada. Por isso sobrescrevemos o método `create()` e usamos `User.objects.create_user()`, que faz o hash automaticamente.

 `accounts/serializers.py`

```
from rest_framework import serializers
from django.contrib.auth.models import User

class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(
        write_only=True,
        min_length=8
    )

    class Meta:
        model = User
        fields = ['username', 'email', 'password']

    def create(self, validated_data):
        # cria o user já com senha criptografada
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data['email'],
            password=validated_data['password']
        )
        return user
```

Por que cada coisa?

- **write_only=True**
A senha entra na requisição, mas nunca volta na resposta. Segurança básica.
- **min_length=8**
Validação automática — falha com 400 se o usuário tentar senha curta.
- **create_user()**
O método que faz o hash da senha. Nunca use `User.objects.create()` para isso.
- **Meta.fields**
Define quais campos entram e voltam. `password` está aqui mas é `write_only`.

A view de registro precisa ser pública

Atenção: configuramos `IsAuthenticated` como padrão global no `settings.py`. Para que qualquer pessoa consiga se cadastrar, precisamos sobrescrever com `permission_classes = [AllowAny]` nesta view específica.

`accounts/views.py`

```
from rest_framework import generics
from rest_framework.permissions import AllowAny
from .serializers import RegisterSerializer

class RegisterView(generics.CreateAPIView):
    serializer_class = RegisterSerializer
    permission_classes = [AllowAny]
```

`accounts/urls.py`

```
from django.urls import path
from .views import RegisterView

urlpatterns = [path('register/', RegisterView.as_view())]
```

Por que `CreateAPIView`?

O DRF tem generic views prontas para casos comuns. `CreateAPIView` aceita só POST e cria um objeto novo — exatamente o que cadastro precisa.

Outras generic views úteis:

<code>ListAPIView</code>	Lista (GET)
<code>RetrieveAPIView</code>	Detalhe (GET por id)
<code>UpdateAPIView</code>	Editar (PUT/PATCH)
<code>DestroyAPIView</code>	Excluir (DELETE)
<code>ListCreateAPIView</code>	Lista + Cria

Rotas de login com JWT

O Simple JWT já entrega duas views prontas: TokenObtainPairView (gera access + refresh no login) e TokenRefreshView (renova o access quando ele expira). Basta plugar no core/urls.py.

 core/urls.py

```
from django.contrib import admin
from django.urls import path, include
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns = [
    path('admin/', admin.site.urls),

    # Autenticação
    path('api/auth/register/',
         include('accounts.urls')),
    path('api/auth/login/',
         TokenObtainPairView.as_view(),
         name='token_obtain_pair'),
    path('api/auth/refresh/',
         TokenRefreshView.as_view(),
         name='token_refresh'),

    # Recurso protegido
    path('api/', include('journal.urls')),
]
```

Endpoints de autenticação

POST

/api/auth/register/

Cria um novo usuário

POST

/api/auth/login/

Retorna access + refresh

POST

/api/auth/refresh/

Renova o access token

PASSO 8 · TESTE

Registrando e fazendo login no Postman

Rode o servidor (python manage.py runserver) e abra o Postman. Vamos primeiro criar um usuário e depois trocar usuário+senha pelo par de tokens.

 **POST** /api/auth/register/

Body → raw → JSON

```
{
  "username": "maria",
  "email": "maria@email.com",
  "password": "senha1234"
}
```

 **Resposta 201 Created**

```
{
  "id": 1,
  "username": "maria",
  "email": "maria@email.com"
}
```

Senha não volta na resposta (write_only).

 **POST** /api/auth/login/

Body → raw → JSON

```
{
  "username": "maria",
  "password": "senha1234"
}
```

 **Resposta 200 OK**

```
{
  "refresh": "eyJ0eXAiOiJKV1Qi...",
  "access": "eyJ0eXAiOiJKV1Qi..."
}
```

Copie o access — você vai usá-lo no header das próximas requests.

Serializer + ViewSet do diário

O author não vem no body — a gente puxa do request.user (que o JWT já preencheu). E no get_queryset filtramos para o usuário só enxergar o que é dele.

`journal/serializers.py`

```
from rest_framework import serializers
from .models import JournalEntry

class JournalEntrySerializer(
    serializers.ModelSerializer
):
    author = serializers.ReadOnlyField(
        source='author.username'
    )

    class Meta:
        model = JournalEntry
        fields = [
            'id', 'author',
            'title', 'content',
            'mood', 'created_at',
        ]
        read_only_fields = [
            'created_at',
        ]
```

`journal/views.py`

```
from rest_framework import viewsets
from rest_framework.permissions
    import IsAuthenticated
from .models import JournalEntry
from .serializers import (
    JournalEntrySerializer,
)

class JournalEntryViewSet(
    viewsets.ModelViewSet
):
    serializer_class = JournalEntrySerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return JournalEntry.objects.filter(
            author=self.request.user
        )

    def perform_create(self, serializer):
        serializer.save(author=self.request.user)
```

IsAuthenticated — só entra quem tem token

Já configuramos IsAuthenticated como permissão padrão no settings.py. Significa que qualquer view sem permission_classes própria exige um JWT válido no header Authorization. Sem token, o DRF responde 401 antes mesmo de chegar na sua view.

AllowAny

Qualquer um, com ou sem login. Usar em registro e endpoints públicos.

```
permission_classes = [AllowAny]
```

IsAuthenticated

Exige token JWT válido. Padrão deste projeto para tudo que envolve dados do usuário.

```
permission_classes =  
[IsAuthenticated]
```

IsAdminUser

Só quem tem is_staff=True. Para áreas administrativas da API.

```
permission_classes =  
[IsAdminUser]
```

IsAuthenticatedOrReadOnly

Leitura para todos, mas só logados podem criar/editar/excluir.

```
permission_classes = [...]
```

💡 Como o padrão já é IsAuthenticated, no nosso JournalEntryViewSet não precisamos nem declarar permission_classes — ele já está protegido. Mas a RegisterView precisou de AllowAny, senão ninguém conseguiria nem se cadastrar (precisaria já estar logado).

IsOwner — só o dono pode editar

Já garantimos que o usuário só vê suas próprias entradas (via `get_queryset`). Mas e se alguém tentar editar pelo ID? Vamos criar uma permissão customizada que checa o dono no nível do objeto.

`journal/permissions.py`

```
from rest_framework import permissions

class IsOwner(permissions.BasePermission):
    """Permite acesso só ao dono do objeto."""

    def has_object_permission(self, request,
                             view, obj):
        return obj.author == request.user
```

`journal/views.py` (atualizar)

```
from .permissions import IsOwner

class JournalEntryViewSet(viewsets.ModelViewSet):
    serializer_class = JournalEntrySerializer
    permission_classes = [IsAuthenticated, IsOwner]
    # ...resto da classe
```

Como funciona

- **BasePermission**
Toda permissão customizada herda dela e implementa um dos dois métodos: `has_permission` ou `has_object_permission`.
- **has_object_permission**
Recebe a `request` e o objeto sendo acessado. Aqui comparamos `obj.author` com `request.user`.
- **Quando é chamado?**
Em GET `/entries/{id}/`, PUT, PATCH e DELETE — operações sobre um objeto específico.
- **E na listagem?**
Em GET `/entries/` (lista), quem filtra é o `get_queryset`. Object permission não roda em listas.

Testando as rotas protegidas

Vamos provar que a autenticação funciona. Mesmo endpoint, três cenários diferentes.

Sem token

401

REQUEST

```
GET /api/entries/  
(sem header Authorization)
```

RESPONSE

```
{  
  "detail": "Authentication  
credentials were not  
provided."  
}
```

Token inválido / expirado

401

REQUEST

```
GET /api/entries/  
Authorization: Bearer XYZ123
```

RESPONSE

```
{  
  "detail": "Given token not  
valid for any token type",  
  "code": "token_not_valid"  
}
```

Token válido

200

REQUEST

```
GET /api/entries/  
Authorization: Bearer <access>
```

RESPONSE

```
[  
  {  
    "id": 1,  
    "title": "Minha 1ª entrada",  
    "author": "wendell"  
  }  
]
```

Permissões por Group (perfis de usuário)

Quando a regra deixa de ser "é meu ou não é" e vira "que tipo de usuário ele é", entram os Groups: você cria perfis (Leitor, Editor, Admin) e atribui usuários a eles. Cada grupo carrega um conjunto de permissões.

LEITOR

Só pode visualizar entradas (próprias).

PERMISSÕES

view_journalentry

EDITOR

Cria, edita e visualiza próprias entradas.

PERMISSÕES

add, change, view

ADMIN

Tudo. Inclusive ver entradas de outros.

PERMISSÕES

add, change, view, delete

 **python manage.py shell**

```
from django.contrib.auth.models import Group, Permission
editor, _ = Group.objects.get_or_create(name='Editor')
perms = Permission.objects.filter(codename__in=['add_journalentry', 'change_journalentry', 'view_journalentry'])
editor.permissions.set(perms)

# Atribuir usuário ao grupo
user.groups.add(editor)
```


Boas práticas antes de subir para produção

01

SECRET_KEY fora do código

Use variáveis de ambiente (python-decouple). Nunca commit no Git.

02

DEBUG = False em produção

Com DEBUG=True qualquer erro vaza traceback, paths e variáveis internas.

03

Senha forte sempre

validators no AUTH_PASSWORD_VALIDATORS. Nunca armazene em texto puro — Django já faz hash.

04

HTTPS obrigatório

Token JWT trafega no header — sem HTTPS qualquer um intercepta a sessão.

05

Tempo de vida curto

Access token de 5-15 min, refresh de poucos dias. Token longo demais = risco maior.

06

ALLOWED_HOSTS restrito

Defina exatamente os domínios. Nunca deixe ['*'] em produção.

💡 Em produção: o comando `python manage.py check --deploy` lista o que ainda falta endurecer no projeto.

Seus desafios da aula

Depois de concluir o passo a passo da Personal Journal API, evolua o projeto com estes 4 desafios. Eles consolidam autenticação, autorização e boas práticas.

01

FÁCIL

Endpoint /api/me/

Crie uma rota que retorna o usuário logado (id, username, email, data de cadastro). Use `IsAuthenticated` e `request.user`.

02

MÉDIO

Compartilhar entrada

Adicione um campo `is_public` ao modelo. Crie uma rota `/api/journal/public/` que lista entradas públicas de todos os usuários, sem precisar de login.

03

MÉDIO

Logout com blacklist

Configure `rest_framework_simplejwt.token_blacklist` e implemente um endpoint `POST /api/auth/logout/` que invalida o refresh token enviado.

04

DIFÍCIL

Permissão de Editor

Use Groups: crie o grupo "Editor". Permita que membros desse grupo vejam (somente leitura) entradas de qualquer usuário, mas não editem nem apaguem.

O que você levou desta aula

Sua API agora distingue usuários e protege dados. Esses são os conceitos que ficam:



Autenticação ≠ Autorização

Autenticar prova quem é.
Autorizar decide o que pode.



JWT

Token assinado que viaja no header em cada requisição.



Permissions DRF

IsAuthenticated,
IsAdminUser e classes
customizadas como
IsOwner.



Groups

Perfis com conjuntos de
permissões para escalar
autorização.

PRÓXIMA AULA

Testes, documentação automática e deploy

Vamos escrever testes para a Journal API, gerar documentação automática com drf-spectacular (Swagger/OpenAPI) e publicar a aplicação na nuvem.